

Práctica 1 — MongoDB CRUD

Operaciones de creación, consulta, actualización y eliminación sobre una base documental

Estudiante	Código SIS	Fecha de entrega

Objetivo

Ejecutar el ciclo CRUD completo de MongoDB desde mongosh sobre una base de datos real, distinguiendo las operaciones que afectan a un documento de las que afectan a muchos, y reconociendo los tres comportamientos de MongoDB que más errores producen: los campos ausentes, los tipos mal cargados y las condiciones sobre arrays de subdocumentos.

Cómo está organizada esta práctica

Parte	Qué contiene	Qué hacés
A · Práctica guiada	32 pasos con los comandos ya escritos y el resultado que debe salir	Los ejecutás y comprobás
B · Ejercicios resolutivos	32 enunciados sin comandos	Los resolvés vos
C · Entrega	Formato de las evidencias y criterios de evaluación	Lo leés antes de empezar

La regla que ordena todo

La Parte A contiene todos los comandos que hacen falta para resolver la Parte B. Ningún ejercicio pide un operador o un método que no se haya usado antes en la guía. Si hacés la Parte A con atención, la Parte B es cuestión de combinar lo que ya viste.

Qué necesitás tener

- MongoDB Community Server instalado y con el servicio corriendo (puerto 27017).
- mongosh, y opcionalmente MongoDB Compass para mirar los datos.
- Las herramientas mongoimport y mongoexport (MongoDB Database Tools).
- Los archivos productos.json, pedidos.json, seed.js y practica1-plantilla.js, en una misma carpeta.

La base de datos de la práctica

Una tienda de artesanías. Base de datos tienda, con dos colecciones:

Colección	Documentos	Qué guarda
productos	12	código, nombre, precio, stock, activo, categoría, arrays de etiquetas y categorías, subdocumento de medidas, array de subdocumentos de inventario por almacén, y la fecha de registro
pedidos	6	cliente, ciudad, estado, array de ítems con código, cantidad y precio, total y fecha

Estos datos tienen tres irregularidades puestas a propósito. No son errores de la práctica: son las situaciones que vas a encontrar en cualquier base documental real, y varios ejercicios dependen de ellas.

Irregularidad	Dónde está	Para qué sirve
Un producto sin el campo precio	TEX-012, Poncho de vicuña	\$exists, ordenar con campos ausentes, \$not, \$mul
stock_minimo cargado como texto	CER-007 y LIB-009	\$type, auditar datos mal cargados
Inventario repartido por almacén	todos los productos	\$elemMatch y su diferencia con la notación de punto

Parte A · Práctica guiada

Esta parte no se inventa nada: los comandos ya están escritos. Tu trabajo es ejecutarlos uno por uno en mongosh, leer la salida y comprobar que coincide con lo que dice «Qué deberías ver». Si algo no coincide, parás y averiguás por qué antes de seguir.

Ejecutá los 32 pasos en orden. De cada bloque tenés que entregar la captura que se indica en la Parte C.

Por qué esta parte es obligatoria

Todos los comandos que necesitás para resolver la Parte B están en la Parte A. No hay ningún ejercicio de la Parte B que pida algo que no se haya mostrado aquí. Si en la Parte B no sabés por dónde empezar, la respuesta está en volver al paso correspondiente de la Parte A.

Bloque 0 · Preparación del entorno

A1 · Comprobar que MongoDB está instalado

Se ejecuta en la terminal de Windows (CMD o PowerShell), NO dentro de mongosh.

```
mongosh --version
```

Qué deberías ver: Un número de versión, por ejemplo 2.3.1. Si dice que el comando no se reconoce, MongoDB no quedó en el PATH: reinstalá marcando la opción o agregá C:\MongoDB\bin al PATH.

A2 · Conectarse al servidor

Sin argumentos, mongosh se conecta solo a localhost:27017.

```
mongosh
```

Qué deberías ver: El prompt cambia a test> y arriba aparece la versión del servidor. Ese prompt indica en qué base estás parado.

A3 · Importar los datos de la práctica

Se ejecuta en la terminal de Windows, parado en la carpeta donde están los .json. NO dentro de mongosh. -d y -c son las abreviaturas de --db y --collection.

```
mongoimport -d tienda -c productos --file productos.json --jsonArray --drop
```

```
mongoimport -d tienda -c pedidos --file pedidos.json --jsonArray --drop
```

Qué deberías ver: 12 document(s) imported successfully. 0 document(s) failed to import. y después lo mismo con 6 documentos. La opción --drop borra la colección antes de importar, así podés repetir el comando sin duplicar datos.

A4 · Verificar la carga

Ahora sí, dentro de mongosh.

```
use tienda
show collections
db.productos.countDocuments() // 12
db.pedidos.countDocuments()   // 6
```

Qué deberías ver: switched to db tienda, después las dos colecciones (pedidos y productos), y los números 12 y 6. Si no salen esos números, no sigas: repetí el paso A3.

A5 • Exportar una colección (el camino inverso)

En la terminal de Windows. Sirve para respaldar o para llevarse los datos a otra máquina.

```
mongoexport -d tienda -c productos --out copia-productos.json --jsonArray
```

Qué deberías ver: exported 12 records. Abrí el archivo generado y mirá cómo MongoDB escribe las fechas: {"\$date": "..."}.

Ese formato se llama JSON extendido.

A6 • Restaurar el punto de partida

Dentro de mongosh, parado en la carpeta donde está seed.js. Este comando lo vas a usar muchas veces.

```
load("seed.js")
```

Qué deberías ver: productos: 12 y pedidos: 6. El script borra las dos colecciones y las vuelve a crear, así que podés romper la base tranquilo: siempre volvé acá.

Bloque 1 • Leer datos

A7 • find() y findOne()

find() devuelve un cursor (un puntero al resultado); findOne() devuelve un documento suelto.

```
db.productos.find()
db.productos.find({})          // idéntico al anterior
db.productos.findOne()
db.productos.find().limit(3)
```

Qué deberías ver: find() lista los 12 productos entre corchetes. findOne() muestra uno solo, sin corchetes. Ya no hace falta .pretty(): mongosh formatea la salida de fábrica.

A8 • Contar y ver valores distintos

```
db.productos.countDocuments()           // 12
db.productos.countDocuments({ activo: true }) // 10
db.productos.distinct("categoria")       // [ 1, 2, 3, 4, 5, 6, 7, 8 ]
db.pedidos.distinct("ciudad")
```

Qué deberías ver: Los números 12 y 10, la lista de categorías, y las cuatro ciudades: Cochabamba, La Paz, Potosí y Sucre.

A9 • Filtrar: igualdad y AND implícito

Escribir dos claves en el mismo objeto ya significa «y».

```
db.productos.find({ categoria: 4 })           // 3 documentos
db.productos.find({ categoria: 4, activo: true }) // 2 documentos
db.productos.find({ activo: false })          // 2 documentos
```

Qué deberías ver: La segunda consulta devuelve uno menos que la primera: el poncho de vicuña queda fuera porque su campo activo es false.

A10 • Notación de punto: entrar en subdocumentos y arrays

El nombre del campo va SIEMPRE entre comillas cuando lleva un punto.

```
db.productos.find({ "medidas.unidad": "cm" })
db.productos.find({ "medidas.alto": { $gt: 100 } }) // 2 documentos
db.pedidos.find({ "items.codigo": "ALM-003" })      // 1 documento
```

Qué deberías ver: La última consulta encuentra un pedido buscando dentro de un array de subdocumentos: no hace falta saber en qué posición está el ítem.

A11 · Proyecciones: elegir qué campos traer

Es el segundo argumento de `find()`. 1 = incluir, 0 = excluir; no se mezclan, salvo `_id`.

```
db.productos.find({ categoria: 1 }, { nombre: 1, precio: 1, _id: 0 })

db.productos.find({ codigo: "ART-001" }, { inventario: 0, medidas: 0 })
```

Qué deberías ver: La primera devuelve tres documentos con solo dos campos cada uno. La segunda devuelve el charango completo menos esos dos campos.

A12 · Ordenar y paginar

`sort`, `limit` y `skip` se encadenan al cursor. 1 = ascendente, -1 = descendente.

```
db.productos.find({}, { nombre: 1, precio: 1, _id: 0 })
    .sort({ precio: -1 })
    .limit(3)

db.productos.find().sort({ precio: 1 }).skip(3).limit(3)    // página 2

db.productos.find().sort({ stock: -1, nombre: 1 })
```

Qué deberías ver: Los tres más caros: Máscara de diablada (1250), Charango (850) y Chompa de alpaca (480). En orden ascendente, el primero de todos es el poncho de vicuña: no tiene precio, y un campo ausente vale menos que cualquier número.

A13 · Filtrar por fechas

Las fechas se escriben con `ISODate()`, no con comillas. Un texto que parece fecha no se puede comparar.

```
db.productos.find({ registrado: { $gte: ISODate("2025-01-01") } })    // 6

db.productos.find({ registrado: {
    $gte: ISODate("2024-01-01"),
    $lt:  ISODate("2025-01-01")
} })                                                                    // 5
```

Qué deberías ver: 6 productos registrados desde 2025 en adelante y 5 registrados durante 2024. Fijate en el patrón para «un año entero»: mayor o igual al 1 de enero, y menor al 1 de enero del año siguiente.

A14 · Comparar un subdocumento completo (y su trampa)

Las dos consultas piden lo mismo en castellano.

```
db.productos.find({ medidas: { alto: 60, ancho: 20, unidad: "cm" } })    // 1

db.productos.find({ medidas: { unidad: "cm", alto: 60, ancho: 20 } })    // 0
```

Qué deberías ver: Una devuelve un documento y la otra ninguno. Al comparar el subdocumento entero, MongoDB exige que las claves estén en el mismo orden. Por eso casi siempre conviene la notación de punto del paso A10.

Bloque 2 · Operadores de consulta

Un operador es una clave que empieza con `$` y ocupa el lugar del valor: { campo: { \$operador: valor } }.

A15 • Comparación: \$gt \$gte \$lt \$lte \$ne

```
db.productos.find({ precio: { $gt: 300 } })           // 4
db.productos.find({ precio: { $gte: 100, $lte: 500 } }) // 5
db.productos.find({ stock: { $lt: 5 } })             // 2
db.productos.find({ activo: { $ne: true } })         // 2
```

Qué deberías ver: Dos operadores sobre el mismo campo se combinan con «y»: la segunda consulta significa «entre 100 y 500».

A16 • \$in y \$nin: varios valores a la vez

Sobre un array, \$in pregunta si ALGUNO de sus elementos está en la lista.

```
db.productos.find({ categoria: { $in: [1, 2] } })     // 4
db.productos.find({ categoria: { $nin: [1, 2] } })    // 8

db.productos.find({ etiquetas: { $in: ["organico", "artesania"] } }) // 5
db.productos.find({ codigo: { $in: [/^ALM/, /^BEB/] } }) // 4
```

Qué deberías ver: 4 y 8 suman 12: todos los productos. \$nin también trae los documentos donde el campo no existe.

A17 • Lógicos: \$or, \$and, \$nor, \$not

\$or, \$and y \$nor reciben un array de condiciones. \$not envuelve a otro operador.

```
// 4 documentos
db.productos.find({ $or: [ { precio: { $lt: 50 } },
                          { stock: { $lt: 5 } } ] })

// 4 documentos
db.productos.find({ $and: [ { activo: true },
                           { precio: { $lt: 100 } } ] })

// 2 documentos
db.productos.find({ $nor: [ { activo: true },
                           { precio: { $gt: 500 } } ] })

// 8 documentos <- ojo con este
db.productos.find({ precio: { $not: { $gt: 300 } } })
```

Qué deberías ver: La última devuelve 8 e incluye el poncho de vicuña, que NO tiene el campo precio. \$not también hace match con los documentos donde el campo no existe: es la causa más común de resultados inesperados.

A18 • \$exists: campos que están o no están

```
db.productos.find({ precio: { $exists: false } })    // 1
db.productos.find({ descuento: { $exists: true } }) // 3
db.productos.find({ variantes: { $exists: true } }) // 2

db.pedidos.find({ "items.1": { $exists: true } })   // 2
```

Qué deberías ver: La última consulta es un truco muy útil: «existe la posición 1 del array» significa «el array tiene al menos dos elementos». Así se filtran los pedidos con más de un ítem.

A19 • \$type: encontrar datos mal cargados

En una base sin esquema, esta es la consulta que más se usa en la vida real.

```
db.productos.find({ stock_minimo: { $type: "string" } }) // 2
db.productos.find({ stock_minimo: { $type: "number" } }) // 10
db.productos.find({ categorias: { $type: "array" } }) // 12
```

Qué deberías ver: Dos productos tienen el stock mínimo cargado como texto ("5" y "10") en vez de número. MongoDB los aceptó sin protestar: el esquema lo cuida quien carga los datos.

A20 • Consultar arrays

Con arrays, «el campo cumple la condición» significa «al menos un elemento la cumple».

```
// contiene ese elemento // 3 docs
db.productos.find({ etiquetas: "artesania" })

// contiene todos estos // 1 doc
db.productos.find({ etiquetas: { $all: ["textil", "artesania"] } })

// cuántos elementos tiene el array // 3 y 5 docs
db.productos.find({ etiquetas: { $size: 2 } })
db.productos.find({ categorias: { $size: 1 } })

// por posición dentro del array // 3 y 9 docs
db.productos.find({ "categorias.0": 4 })
db.productos.find({ "etiquetas.2": { $exists: true } })
```

Qué deberías ver: La primera consulta no compara el array con un texto: pregunta si alguno de sus elementos es "artesania".

A21 • \$elemMatch: la trampa clásica

Pregunta: ¿qué productos tienen menos de 10 unidades en el almacén de Sucre?

```
// Sin $elemMatch – devuelve 3, y uno sobra
db.productos.find({
  "inventario.almacen": "Sucre",
  "inventario.cantidad": { $lt: 10 }
})

// Con $elemMatch – devuelve 2, correcto
db.productos.find({
  inventario: { $elemMatch: { almacen: "Sucre", cantidad: { $lt: 10 } } }
})
```

Qué deberías ver: La primera cuela a LIB-009, que tiene Sucre con 20 y Potosí con 6: una condición la cumple un elemento del array y la otra, otro distinto. \$elemMatch exige que sea el MISMO elemento el que cumpla todo. Regla: array de subdocumentos + condición sobre dos campos ⇒ \$elemMatch.

A22 · \$regex: buscar por texto

Anclar con ^ permite usar un índice; sin ancla se recorre toda la colección.

```
db.productos.find({ nombre: { $regex: "^Ch" } }) // 3
db.productos.find({ nombre: { $regex: "plata", $options: "i" } }) // 1
db.productos.find({ codigo: { $regex: "^ALM-" } }) // 3

db.productos.find({ nombre: /^[AC]/ }) // 6
```

Qué deberías ver: La última usa la sintaxis corta de JavaScript y los corchetes de las expresiones regulares: «que empiece con A o con C».

Bloque 3 · Escribir datos

Antes de empezar este bloque

Ejecutá load("seed.js"). Los pasos que siguen modifican la base, y los números de «Qué deberías ver» suponen los 12 productos y 6 pedidos originales.

A23 · insertOne(): un documento

```
db.productos.insertOne({
  codigo: "BEB-013",
  nombre: "Chuflay embotellado 500 ml",
  precio: 75,
  stock: 20,
  activo: true,
  categoria: 2,
  etiquetas: [ "bebida", "tarija" ]
})
```

Qué deberías ver: acknowledged: true y un insertedId con un ObjectId. Si la colección no existiera, se crearía en ese momento.

A24 · insertMany(): varios de golpe

```
db.productos.insertMany([
  { codigo: "ALM-014", nombre: "Maní de Chuquisaca 500 g",
    precio: 35, stock: 80 },
  { codigo: "ALM-015", nombre: "Api morado en polvo",
    precio: 18, stock: 120 }
])

db.productos.countDocuments() // 15
```

Qué deberías ver: insertedIds con dos ObjectId, y el total en 15. Notá que esos dos productos no tienen activo, ni categorías, ni etiquetas: MongoDB no protesta.

A25 · updateOne() y \$set

\$set crea el campo si no existía. El segundo argumento SIEMPRE lleva un operador.

```
load("seed.js")

db.pedidos.updateOne(
  { _id: 1 },
  { $set: { estado: "enviado", transportadora: "Trans Sucre" } }
)
```

Qué deberías ver: matchedCount: 1 y modifiedCount: 1. El pedido no tenía transportadora y ahora la tiene. Si escribís el segundo argumento sin operador, mongosh devuelve un error.

A26 · updateMany() y \$inc

matchedCount dice a cuántos llegó el filtro; modifiedCount, a cuántos les cambió algo.

```
db.productos.updateMany({ categoria: 1 }, { $inc: { precio: 5 } })

db.productos.updateOne({ codigo: "ART-001" }, { $inc: { stock: -1 } })
```

Qué deberías ver: matchedCount: 3, modifiedCount: 3. Los alimentos pasan de 45 a 50, de 28 a 33 y de 60 a 65. \$inc con un número negativo resta.

A27 · \$mul, \$rename y \$currentDate

```
load("seed.js")

db.productos.updateMany({ categoria: 4 }, { $mul: { precio: 1.1 } })
db.productos.find({ categoria: 4 }, { codigo: 1, precio: 1, _id: 0 })

db.pedidos.updateMany({ estado: "enviado" },
  { $set: { estado: "entregado" }, $currentDate: { fecha_entrega: true } })
```

Qué deberías ver: El aguayo pasa de 320 a 352 y la chompa de 480 a 528. Pero mirá el poncho de vicuña: no tenía precio y ahora tiene precio 0. \$mul sobre un campo que no existe lo crea en cero. Es un comportamiento documentado y una fuente clásica de errores.

A28 · \$unset: borrar un campo

```
load("seed.js")

db.productos.updateOne({ codigo: "ART-001" },
  { $unset: { stock_minimo: "" } })
db.productos.find({ codigo: "ART-001" })
```

Qué deberías ver: El documento ya no tiene el campo stock_minimo: no queda en null, desaparece. Después de un \$unset, { stock_minimo: { \$exists: false } } lo encuentra.

A29 · Modificar arrays: \$push, \$addToSet, \$pull, \$pop, \$each

```
db.productos.updateOne({ codigo: "ALM-003" },
  { $push: { etiquetas: "promocion" } })

db.productos.updateOne({ codigo: "ALM-003" },
  { $addToSet: { etiquetas: "organico" } }) // ya está: modified 0

db.productos.updateOne({ codigo: "ALM-003" },
  { $pull: { etiquetas: "grano" } })

db.productos.updateOne({ codigo: "ALM-003" },
  { $push: { etiquetas: { $each: ["oferta", "2x1"] } } })

db.productos.updateOne({ codigo: "ALM-003" },
  { $pop: { etiquetas: 1 } })

// también funciona con subdocumentos dentro del array
db.productos.updateOne({ codigo: "ART-001" },
  { $push: { inventario: { almacen: "Tarija", cantidad: 4 } } })
```

Qué deberías ver: El \$addToSet devuelve modifiedCount: 0 porque "organico" ya estaba: esa es toda la diferencia con \$push. \$pop con 1 quita el último elemento; con -1, el primero.

A30 · replaceOne(): sustituye, no completa

```
load("seed.js")

db.productos.replaceOne(
  { codigo: "CER-007" },
  { codigo: "CER-007", nombre: "Vasija de cerámica Tiwanaku", precio: 260 }
)
db.productos.find({ codigo: "CER-007" })
```

Qué deberías ver: El documento queda con tres campos y el _id. Perdió stock, activo, categoria, categorias, etiquetas, medidas, inventario y registrado. replaceOne reemplaza todo lo que no sea el _id.

A31 · upsert: actualizar si existe, insertar si no

```
db.productos.updateOne(
  { codigo: "BEB-099" },
  { $set: { nombre: "Producto de prueba", precio: 10, stock: 1 } },
  { upsert: true }
)
```

Qué deberías ver: matchedCount: 0, modifiedCount: 0, upsertedCount: 1 y un upsertedId. El documento insertado combina el filtro con el \$set: por eso queda con codigo, nombre, precio y stock.

A32 · Borrar: deleteOne, deleteMany y drop

No hay papelera ni ROLLBACK. El hábito correcto es contar primero.

```
load("seed.js")

db.pedidos.countDocuments({ estado: "enviado" }) // 2 ← reviso primero
db.pedidos.deleteMany({ estado: "enviado" })    // ahora sí

db.pedidos.deleteOne({ estado: "pendiente" })   // borra UNO solo

// db.pedidos.drop()           borra la colección entera y sus índices
// db.dropDatabase()          borra la base entera

load("seed.js") // dejamos todo como estaba
```

Qué deberías ver: deletedCount: 2 y después deletedCount: 1. Los dos últimos comandos están comentados a propósito: no los ejecutes todavía. Terminá siempre restaurando con seed.js.

Parte B · Ejercicios resolutivos

Acá no hay comandos escritos: los escribís vos. Todo lo que necesitás está en la Parte A — si un ejercicio te trava, volvé al paso de la guía que trata ese operador.

Cada respuesta va en el archivo **practica1-TUAPELLIDO.js**, debajo del comentario del enunciado que corresponde. Los ejercicios que piden una explicación se responden en un comentario `//` al lado del comando.

Antes de empezar

Ejecutá `load("seed.js")`. La plantilla ya trae esa línea al principio y otras dos en el medio, antes de las secciones de actualización y de eliminación: no las borres, son las que hacen que tus resultados coincidan con los esperados.

B1 – B18 · Consultas

- B1.** Mostrá los productos de la categoría 2 o 7, con el nombre y el precio únicamente, sin el `_id`.
- B2.** Mostrá los productos cuyo precio esté entre 100 y 300, incluidos los dos extremos.
- B3.** Mostrá los productos que no están activos.
- B4.** Mostrá los productos cuyo nombre empiece con la letra A o con la letra C.
- B5.** Mostrá los productos que tienen el campo `variantes`.
- B6.** Encontrá los productos donde `stock_minimo` se cargó como texto en vez de número.
- B7.** Mostrá los 4 productos con más stock, con el nombre y el stock únicamente.
- B8.** Mostrá la segunda página de un listado ordenado por nombre ascendente, de 4 en 4.
Pista: la primera página serían los 4 primeros; la segunda empieza en el quinto.
- B9.** Mostrá los productos que tengan la etiqueta "organico" o la etiqueta "artesania".
Pista: una sola condición alcanza; no hace falta \$or.
- B10.** Mostrá los productos cuyo array `categorias` tenga exactamente un elemento.
- B11.** Mostrá los productos con menos de 10 unidades en el almacén de La Paz. Escribí las dos versiones —sin `$elemMatch` y con `$elemMatch`— y anotá cuántos documentos devuelve cada una y por qué son distintas.
Pista: una de las dos devuelve un producto de más; identificá cuál es y mirá su inventario.
- B12.** Mostrá los productos cuya primera categoría del array `categorias` sea 1.
- B13.** Mostrá los productos registrados durante el año 2025.
- B14.** Averiguá cuántos productos están activos. Se pide el número, no la lista.
- B15.** Mostrá los pedidos de la ciudad de Sucre cuyo total sea mayor a 300.
- B16.** Mostrá los pedidos que incluyan el producto de código "ALM-005".
Pista: el código está dentro de un array de subdocumentos.
- B17.** Mostrá los pedidos que tengan más de un ítem.
Pista: no existe un operador «longitud mayor que»; pensá en la posición 1 del array.
- B18.** Mostrá la lista de clientes distintos que hicieron pedidos.

B19 – B22 · Creación

B19. Insertá un producto nuevo que tenga, como mínimo: un array de textos, un subdocumento y un array de subdocumentos. Los datos son de tu invención, pero tienen que ser coherentes con los demás productos.

B20. Insertá tres productos más en una sola instrucción.

B21. Insertá un pedido con `_id` 7, de un cliente que no exista todavía, con dos ítems.

B22. Insertá un producto sin el campo `precio`. Después contá cuántos productos no tienen `precio` y explicá el número que sale.

B23 – B28 · Actualización

B23. Subí un 10 % el precio de los productos de la categoría 4. Después mirá el precio del poncho de vicuña (TEX-012) y explicá qué le pasó y por qué.

Pista: el poncho no tenía precio antes de la operación.

B24. Pasá a "entregado" todos los pedidos que estén en "enviado", y dejales registrada la fecha de entrega con la hora del servidor. Una sola instrucción.

B25. Agregá la etiqueta "liquidacion" a todos los productos inactivos, de manera que no se duplique si alguno ya la tuviera.

B26. Borrá el campo `stock_minimo` únicamente en los productos donde está cargado como texto. Verificá después que no queda ninguno así.

B27. Agregá al café de los Yungas (ALM-011) un almacén nuevo: "Camiri" con 5 unidades, sin tocar los almacenes que ya tiene.

B28. Con una sola instrucción, actualizá el producto de código "BEB-030" si existe y creálo si no existe, con un nombre, precio 40 y stock 25.

B29 – B32 · Eliminación

B29. Contá cuántos pedidos están cancelados y recién después borralos. Entregá los dos comandos, en ese orden.

B30. Eliminá un solo producto que tenga la etiqueta "textil". Ojo: hay más de uno, y el comando tiene que borrar exactamente uno.

B31. Eliminá los productos con stock menor a 5.

B32. Restaurá la base al punto de partida y verificá que quedaron 12 productos y 6 pedidos.

Parte C · Qué se entrega

La entrega son evidencias de que ejecutaste los comandos, no solo de que los escribiste. Se entregan tres cosas.

1 · El script

Archivo **practica1-TUAPELLIDO.js**, partiendo de la plantilla que se entrega junto con esta práctica. Tiene que poder ejecutarse de principio a fin sin errores:

```
mongosh < practica1-TUAPELLIDO.js
```

Probalo antes de entregar. Si el archivo se corta con un error a la mitad, la práctica se considera no entregada.

2 · El PDF de evidencias

Un PDF donde, para cada ejercicio de la Parte B, aparezca:

- el número y el enunciado,
- el comando que usaste,
- una captura de pantalla de mongosh mostrando ese comando y su salida real.

La captura tiene que incluir el prompt (tienda>) y la respuesta completa. No sirve una captura recortada donde solo se vea el comando.

De la Parte A alcanza con una captura por bloque: la del último paso de cada uno (A6, A14, A22 y A32).

3 · El repositorio

Subí el .js y el PDF a tu repositorio, en la carpeta **practicas/practica1/**. El PDF además se sube al classroom.

Criterios de evaluación

Criterio	Qué se mira	Puntos
Parte A ejecutada	Las 4 capturas de cierre de bloque, con salida coherente	15
Consultas (B1–B18)	Comando correcto y resultado que coincide con los datos	30
Creación (B19–B22)	Documentos coherentes, con array, subdocumento y array de subdocumentos	10
Actualización (B23–B28)	Operador correcto; distinguir updateOne de updateMany	15
Eliminación (B29–B32)	Contar antes de borrar; distinguir deleteOne de deleteMany	10
Explicaciones pedidas	B11, B22 y B23 piden justificar. Sin justificación no puntúan	10
El script corre entero	mongosh < archivo.js termina sin errores	10
	Total	100

Los tres ejercicios que definen la nota

B11 (\$elemMatch), B22 (\$exists sobre un campo ausente) y B23 (\$mul sobre un campo que no existe) son los que separan a quien copió comandos de quien entendió qué hace MongoDB con los datos. Los tres piden una explicación escrita, y los tres se preguntan en la defensa.

Anexo · Chuleta de operadores

Grupo	Operadores
Comparación	<code>\$eq</code> <code>\$ne</code> <code>\$gt</code> <code>\$gte</code> <code>\$lt</code> <code>\$lte</code> <code>\$in</code> <code>\$nin</code>
Lógicos	<code>\$and</code> <code>\$or</code> <code>\$not</code> <code>\$nor</code>
Elemento	<code>\$exists</code> <code>\$type</code>
Arrays (consulta)	<code>\$all</code> <code>\$elemMatch</code> <code>\$size</code>
Evaluación	<code>\$regex</code> <code>\$expr</code> <code>\$mod</code> <code>\$text</code>
Campos (escritura)	<code>\$set</code> <code>\$unset</code> <code>\$inc</code> <code>\$mul</code> <code>\$rename</code> <code>\$currentDate</code> <code>\$min</code> <code>\$max</code>
Arrays (escritura)	<code>\$push</code> <code>\$pop</code> <code>\$pull</code> <code>\$addToSet</code> <code>\$each</code>
Cursor	<code>.sort()</code> <code>.limit()</code> <code>.skip()</code>

Operación	Métodos
Crear	<code>insertOne()</code> <code>insertMany()</code>
Leer	<code>find()</code> <code>findOne()</code> <code>countDocuments()</code> <code>distinct()</code>
Actualizar	<code>updateOne()</code> <code>updateMany()</code> <code>replaceOne()</code>
Borrar	<code>deleteOne()</code> <code>deleteMany()</code> <code>drop()</code> <code>dropDatabase()</code>
Importar / exportar	<code>mongoimport</code> <code>mongoexport</code> (desde la terminal, no desde mongosh)

Código que vas a encontrar en internet y que hoy ya no funciona

`insert()`, `update()`, `remove()` y `save()` fueron retirados: usá `insertOne/insertMany`, `updateOne/updateMany`, `deleteOne/deleteMany` y `upsert`.

`.pretty()` ya no hace nada: `mongosh` formatea solo.

El comando `mongo` se retiró en la versión 5.0; el shell actual es `mongosh`.

Si un tutorial usa `save()` o `update()`, es anterior a 2021.